

Understanding the Intel DPDK Framework

R&D Report

by

Mitali Yadav

Roll No. 143059006

Under the guidance of
Prof. Mythili Vutukuru



Department of Computer Science and Engineering,
Indian Institute of Technology Bombay.
May 2016.

Contents

List of Figures	ii
List of Tables	iii
1 Introduction	1
1.1 Problem Statement	1
1.2 Background and challenges	1
2 DPDK (Data Plane Development Kit)	2
3 Experiment design and challenges	4
3.1 Setup at Host Hide	6
3.2 Setup at Traffic Generator	7
3.3 Setup at VM with DPDK	9
3.4 Setup at VM without DPDK	12
3.5 Running Experiment	14
4 Results	15
5 Conclusion	17
References	18

List of Figures

Figure 2.1	Packet I/O path in DPDK	3
Figure 3.1	System setup for performing experiment	4
Figure 3.2	System setup (with modified L2 forwarder)	5
Figure 3.3	Adding VF as PCIe Virtual functions inside VM	7
Figure 3.4	Setting CPU flags for VM	10
Figure 3.5	Timer code change	11
Figure 3.6	UDP server code	13
Figure 4.1	No. of Packets sent vs Receive Throughput	15
Figure 4.2	No. of Packets sent vs Receive side CPU utilization	16

List of Tables

4.1	Performance results	16
-----	-------------------------------	----

Chapter 1

Introduction

Virtualization plays an important role in current data centers. Although it offers many benefits like flexibility in application deployment, isolation of these application, etc but it also create overhead like VMM intervention for each packet I/O, interrupt destined for VM causes multiple VM exits, etc.

In this R&D, we are exploring Intel DPDK (Data Plane Development Kit) framework which tries to skip OS network stack of VM and delivers packet directly to userspace application present inside VM.

1.1 Problem Statement

To use DPDK inside VM and compare the performance benefit of DPDK based VM to non-DPDK based VM.

1.2 Background and challenges

Chapter-2 gives details on DPDK framework and how it works.

Chapter-3 gives details on experiment design and challenges we overcome during installation of DPDK and running the experiment.

Chapter-4 shows the results of experiment performed.

Chapter-5 concludes that use of DPDK gives performance benefit over regular OS network stack processing inside VM.

Chapter 2

DPDK (Data Plane Development Kit)

The Intel DPDK [1], [2], [3] is a collection of libraries, which try to reduce virtualization overheads by allowing user space application to directly poll the NIC for data. It provides a generic interface to its application irrespective of actual hardware and software architecture via EAL (Environmental Abstraction Layer).

It also uses Linux huge pages to pre-allocate large regions of memory, and then allows applications to DMA data directly into these pages. The poll mode driver allows applications to access the NIC card directly without involving kernel processing, while the buffer and ring management systems resemble the memory management systems employed within the kernel for holding packet buffers.

Here is how packet I/O happens in DPDK (See Figure 2.1):

1. The application (who wants to read the packet) checks the DPDK ring structure (which maintains pointer to packet buffers) for new packets via `dpdk` library function.
2. If new packet descriptors are available in the ring, application access the packet buffers in DPDK mempool area via pointers present in packet descriptor.
3. If no packet are available, applications polls the NIC via poll mode drivers library function and then the ring structure is accessed again.

Note that the driver of DPDK does not work in a transparent mode, so as soon as this driver is loaded, the NIC becomes available to DPDK instead of OS network stack regardless of whether any DPDK-enabled application is running or not.

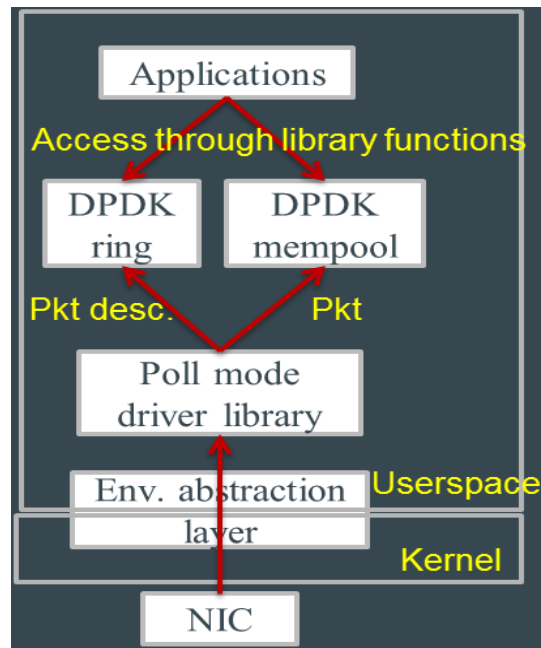


Figure 2.1: Packet I/O path in DPDK

Pros:

- No system call used implies no context switching.
- Direct NIC access to user space applications.

Cons:

- As it uses custom API, familiarity of API is needed to use DPDK.

Chapter 3

Experiment design and challenges

To start with DPDK framework, we first start studying L2 forwarder, how it works and so on. So our first experiment setup was using l2 forwarder without any modification (See Figure 3.1 shows the experiment setup (source [4])).).

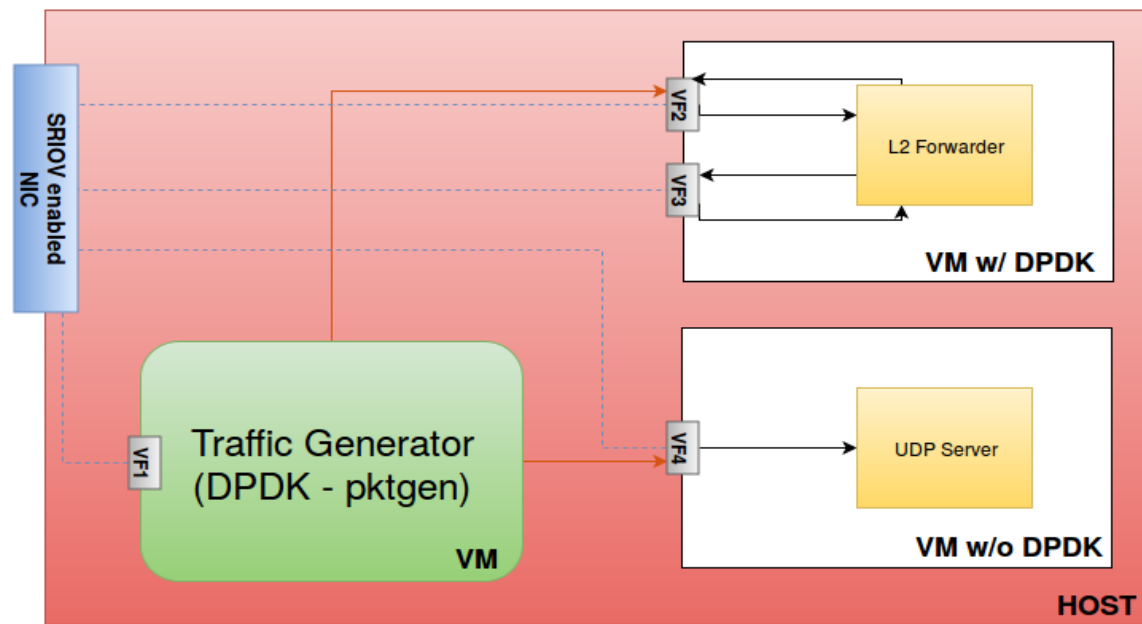


Figure 3.1: System setup for performing experiment

As we are going to measure receive throughput only, we modified L2 forwarder to do the following: receive the incoming packet, measure throughput and then drop it and lastly print the statistics per second.

Figure 3.2 shows the experiment setup with aforementioned changes.

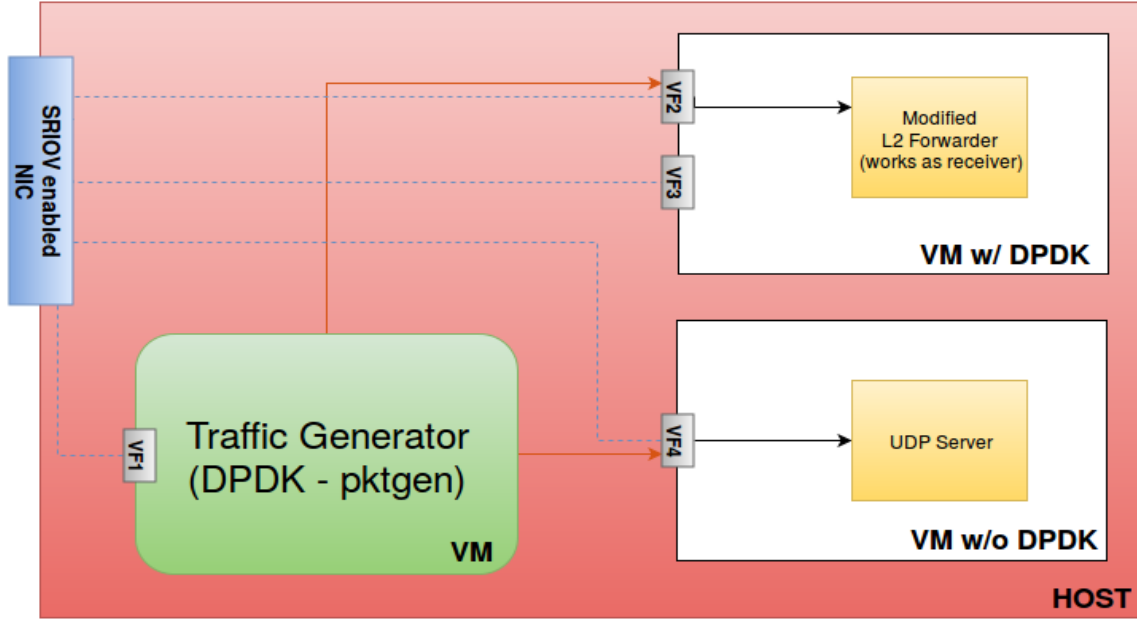


Figure 3.2: System setup (with modified L2 forwarder)

Explanation to system setup:

- This setup is done on one physical machine with Intel(R) Xeon(R) CPU (2.60GHz) and SRIOV enabled NIC (I350 Gigabit NIC).
- There are three VMs created in the host machine. One for generating traffic, one VM with DPDK application running on it and last VM on which a UDP server is running which works as receiver.
- Host uses SRIOV enabled NIC and create 4 virtual functions(VF)¹. One VF is given to traffic generator, two VF to VM w/ DPDK and last one to VM w/o DPDK.
- With the use of VFs, VMM intervention in network I/O is skipped. Hence VMM will not create any bottleneck in performance measurement.
- We have used 'pktgen' as traffic generator.

¹VF are virtual form of physical NIC and able to perform transmit and receive w/o physical NIC involvement.

- VM with DPDK uses L2 forwarder sample example provided by DPDK framework and modify it to work as DPDK receiver.
- VM without DPDK uses a UDP server as a receiver application.

Note: Both DPDK receiver and UDP server is userspace application.

Next we will see instruction to setup the environment shown in Figure-3.2.

3.1 Setup at Host Side

Here at host side, we have a SRIOV enabled physical NIC and we need to create four VFs from it as mentioned earlier in explanation and assign them to three VMs.

For creating and assigning VF to each VM, follow these steps:

1. Enable VT-d and Virtualization support in BIOS.
2. Grub kernel configuration:
 - Open grub config file
 - Add `intel_iommu=on`
3. Following is the command used to create 4 VFs:


```
echo 4 > /sys/class/net/eth0/device/sriov_numvfs
```

Note: This instruction needs super user permission to execute and also in place of 'eth0' use the interface name you are using for experiment.
4. Check whether VFs are created successfully or not:


```
lspci — grep Ethernet
```

Results will be like this:

```
07:00.0 Ethernet controller: Intel Corporation I350 Gigabit Network Connection (rev 01)
08:10.0 Ethernet controller: Intel Corporation I350 Ethernet Controller Virtual Function (rev 01)
08:10.4 Ethernet controller: Intel Corporation I350 Ethernet Controller Virtual Function (rev 01)
08:11.0 Ethernet controller: Intel Corporation I350 Ethernet Controller Virtual Function (rev 01)
08:11.4 Ethernet controller: Intel Corporation I350 Ethernet Controller Virtual Function (rev 01)
```
5. Next is to assign these created VFs to VMs. Follow these two steps to add these VFs as PCIe in VMs:

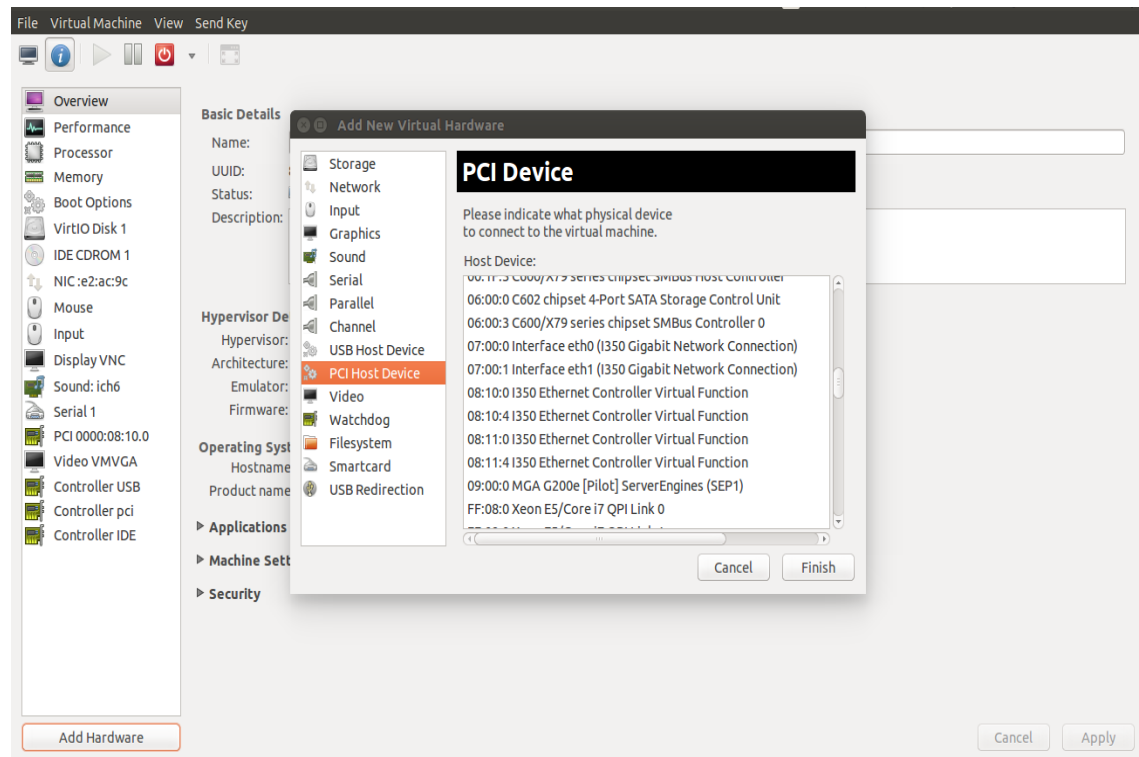


Figure 3.3: Adding VF as PCIe Virtual functions inside VM

Go to VM setting and click on 'Add Hardware' and then go to PCI host device and add respective VF as shown in Figure 3.3

6. There are four VFs, out of which one will be added to VM which is traffic generator, two VFs will be added to VM with DPDK and last one will be added to VM without DPDK.

3.2 Setup at Traffic Generator

We are using 'pktgen' as traffic generator which is a DPDK application so to install pktgen we need to have DPDK framework installed on hand. See section 3.3 for DPDK installation.

After DPDK is successfully installed. Follows these steps to install 'pktgen':

- Download pktgen from [5] or clone its code by following commands:
`git clone git://dpdk.org/apps/pktgen-dpdk`
or

```
git clone http://dpdk.org/git/apps/pktgen-dpdk
```

DPDKInstallDir and PktgenInstallDir are referred as installation directories of dpdk and pktgen respectively.

- Set up the environmental variables required by DPDK:
export RTE_SDK=<DPDKInstallDir>
export RTE_TARGET=x86_64-native-linuxapp-gcc
- Create the DPDK build tree:
cd \$RTE_SDK
make install T=x86_64-native-linuxapp-gcc
- Build pktgen as follows:
cd <PktgenInstallDir>
make
- Following command configure the system to run a DPDK application:
cd <PktgenInstallDir>
sudo ./setup.sh
- Load uio and igb-uio module using following commands:
modprobe uio
insmod <DPDKInstallDir>/x86_64-native-linuxapp-gcc/kmod/igb-uio.ko
- Now following steps are used to generate traffic:
 1. Load pktgen module:
modprobe pktgen
 2. Remove previously added devices pktgen thread 0:
echo "rem_device_all" > /proc/net/pktgen/kpktgend_0
Note: On loading pktgen module, one execution thread per cpu is created.
 3. Add 'eth1' device to pktgen thread 0:
echo "add_device eth1" > /proc/net/pktgen/kpktgend_0
 4. Set packet count as 10 million:
echo "count 10000000" > /proc/net/pktgen/eth1
 5. clone skbuffer for each 10 million packet:
echo "clone_skb 10000000" > /proc/net/pktgen/eth1
 6. Set packet size as 60:
echo "pkt_size 60" > /proc/net/pktgen/eth1

7. Set delay as zero:
`echo "delay 0" > /proc/net/pktgen/eth1`
8. Set destination IP address:
`echo "dst dst_IP_addr" > /proc/net/pktgen/eth1`
9. Set destination MAC address:
`echo "dst_mac dst_mac_addr" > /proc/net/pktgen/eth1`
10. Set minimum destination UDP port address:
`echo "udp_dst_min 12000" > /proc/net/pktgen/eth1`
11. Set maximum destination UDP port address:
`echo "udp_dst_max 12001" > /proc/net/pktgen/eth1`
 It will send packets to all port starting from min port till max-1.
12. This will start traffic generation:
`echo "start" > /proc/net/pktgen/pgctrl`

3.3 Setup at VM with DPDK

Here we first need to install DPDK framework inside VM. These are the step to follow:

- System Requirement:
 Hugepage support:
 – Changes number of huge pages:
`echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages`
 – Reserve huge page memory:
`mkdir /mnt/huge`
`mount -t hugetlbfs nodev /mnt/huge`
 – Mount point can be made permanent across reboots, by opening `/etc/fstab` and add this line at the end:
`nodev /mnt/huge hugetlbfs defaults 0 0`
- Download current version DPDK source code from "<http://dpdk.org/download>"
- Installation:
 – `unzip DPDK-<version>.zip`
 – `cd DPDK-<version>`
 – `make install T=x86_64-native-linuxapp-gcc`
 Here we faced a problem like this:
`"cpu you selected does not support x86-64 instruction set"`
 After searching on this problem, we find out that some cpu flags were missing in VM setting. These flags are `ssse3`, `aes`, `vmx`.

See figure 3.4, open VM setting then go to processor and open "configuration" option. After that click on "Copy host CPU configuration". It will simply copy all host configuration to VM and all required flags will be set.

Now again repeat that make command and after that dpdk build is complete

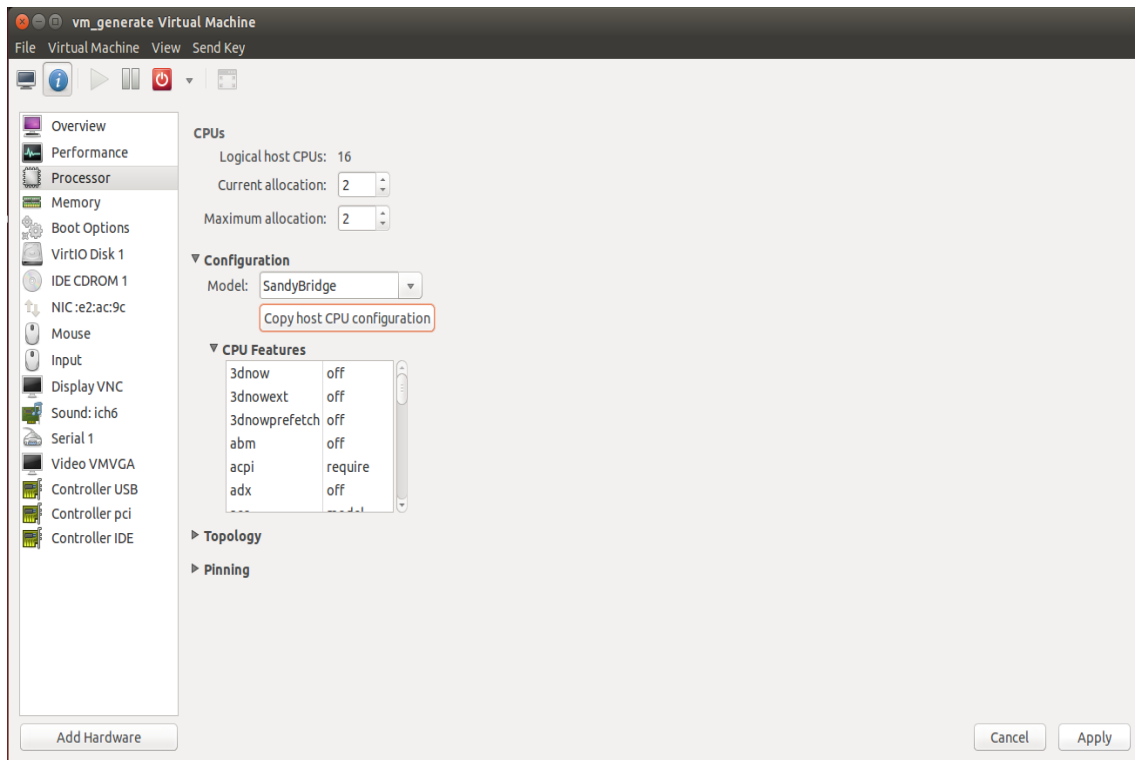


Figure 3.4: Setting CPU flags for VM

and ready for use.

- Now next step is to change L2 forwarder as DPDK receiver:
See [4] for more details on L2 forwarder. Here is the changes done to the code.
 1. Open file DPDK-<version>/examples/l2fwd/main.c
 2. Added header:
`#include <time.h>`
 3. In `print_stats()` method, add these three lines (with + sign) inside for loop after this line:
`total_packets_rx += port_statistics[portid].rx;`
`+port_statistics[portid].tx=0;`

```
+port_statistics[portid].rx=0;
+port_statistics[portid].dropped=0;
This is to clear statistics after each second.
```

4. Inside main loop, comment this line:
`l2fwd_dst_ports[portid] = last_port;`
`+//l2fwd_dst_ports[last_port] = portid;`
 In modified l2 forwarder, we only want to calculate receive throughput. So this will remove unnecessary forwarding operation and causes packet drop operation when VM receive any packet on other VFs.
5. L2 forwarder uses timer based interrupt for printing statistics. This was consuming lots of CPU cycles and creating overhead for receiver. So we replace it (comment the timer based code see figure 3.5) by very simple statistic printing method.
 In `l2fwd_main_loop()` method add following two variable and do to changes shown in figure 3.5:
`clock_t beforebyM,secbyM;`

```
//added be me//
secbyM = (clock() - beforebyM)/CLOCKS_PER_SEC;
if(secbyM>=1){
    if (lcore_id == rte_get_master_lcore()) {
        print_stats();
        beforebyM = clock();
    }
}

/* if timer is enabled */
// if (timer_period > 0) {

    /* advance the timer */
    // timer_tsc += diff_tsc;

    /* if timer has reached its timeout */
    // if (unlikely(timer_tsc >= (uint64_t) timer_period)) {

        /* do this only on master core */
        // if (lcore_id == rte_get_master_lcore()) {
        //     print_stats();
        //     /* reset the timer */
        //     timer_tsc = 0;
        // }
    // }
// }
```

Figure 3.5: Timer code change

6. After doing these changes compile main.c using:
 - cd DPDK-<version>/examples/l2fwd/
 - make
- Now our DPDK receiver is ready. To execute this receiver follow these steps:
 1. modprobe uio
 2. insmod DPDK-<version>/x86_64-native-linuxapp-gcc/kmod/igb_uio.ko
 3. Bind both VFs using:
 - ./tools/dpdk_nic_bind.py –bind=igb_uio 00:07.0
 - ./tools/dpdk_nic_bind.py –bind=igb_uio 00:08.0
 4. Use the following command to run this modified l2 forwarder:
 - DPDK-<version>/examples/l2fwd/build/app/l2fwd -c 3 -n 1 -b 000:00:03.0
 - -q 1 -p 3*

3.4 Setup at VM without DPDK

Here we are creating a simple UDP server as receiver which reads maximum of 2048 bytes if any data available for reading and print the statistics per second. See figure 3.6 for server code (Source [6])

Note: Replace server IP address shown in figure 3.6 by your machine IP address.


```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <error.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>

#define MYPOR 12000
#define MAXBUFL 2048

int main(int argc, char *argv[]){
    int sockfd;
    struct sockaddr_in my_addr;
    struct sockaddr_in their_addr;
    int addr_len, numbytes;
    int packets=0, sec=0;
    char buf[MAXBUFL];
    clock_t before,diff;

    if((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) == -1){
        perror("Server-socket() sockfd error");
        exit(1);
    }else{
        printf("Server-socket() sockfd is OK.\n");
    }

    my_addr.sin_family = AF_INET;
    my_addr.sin_port = htons(MYPOR);
    my_addr.sin_addr.s_addr = inet_addr("10.129.26.210");

    memset(&(my_addr.sin_zero), '\0', 8);

    if(bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct sockaddr)) == -1){
        perror("Server-bind() error");
        exit(1);
    }
    else{
        printf("Server-bind() is OK.\n");
    }

    printf("SERVER started at 12000");

    before = clock();
    while(1){
        sec = (clock() - before)/CLOCKS_PER_SEC;
        if(sec>=1){
            printf("\n-----\npacket rate %d\n-----\n\n",packets);
            packets=0;
            before = clock();
        }

        if((numbytes = recvfrom(sockfd, buf, MAXBUFL, 0, (struct sockaddr *)&their_addr, &addr_len))== -1)
        {
            perror("Server-recvfrom() error");
            exit(1);
        }
        packets++;
    }
    close(sockfd);
    return 0;
}

```

Figure 3.6: UDP server code

Lastly to execute this code do the following:

```
gcc udpserver.c -o server
```

3.5 Running Experiment

Now our system is ready for performing experiment.

Testing VM with DPDK:

- In VM with DPDK, start DPDK receiver.
- On traffic generator VM, set appropriate destination IP and MAC address of VM with DPDK and start generating traffic.
- Note the statistic printed on screen of VM with DPDK.

Testing VM without DPDK:

- In VM without DPDK, start UDP server.
- On traffic generator VM, set appropriate destination IP and MAC address of VM without DPDK and start generating traffic.
- Note the statistic printed on screen of VM without DPDK.

Lastly use top command to see the CPU utilization inside VMs with and without DPDK.

Chapter 4

Results

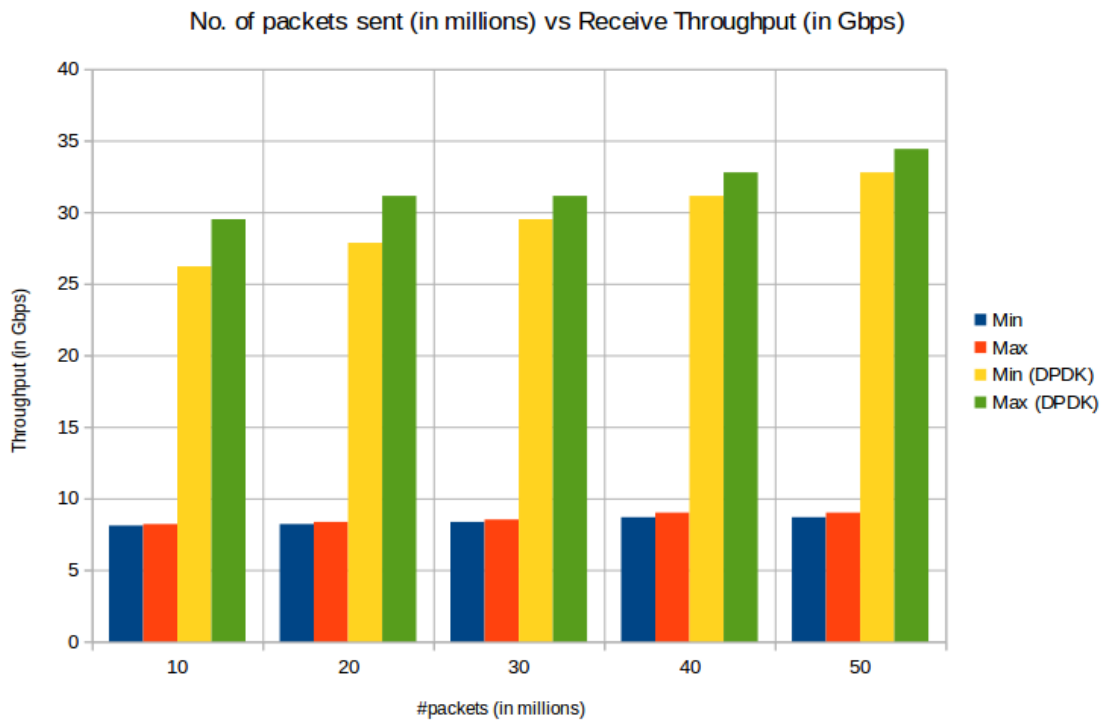


Figure 4.1: No. of Packets sent vs Receive Throughput

We can see from Figure 4.1 VM with DPDK gives more than 50% increase in the performance for packet I/O than VM without DPDK.

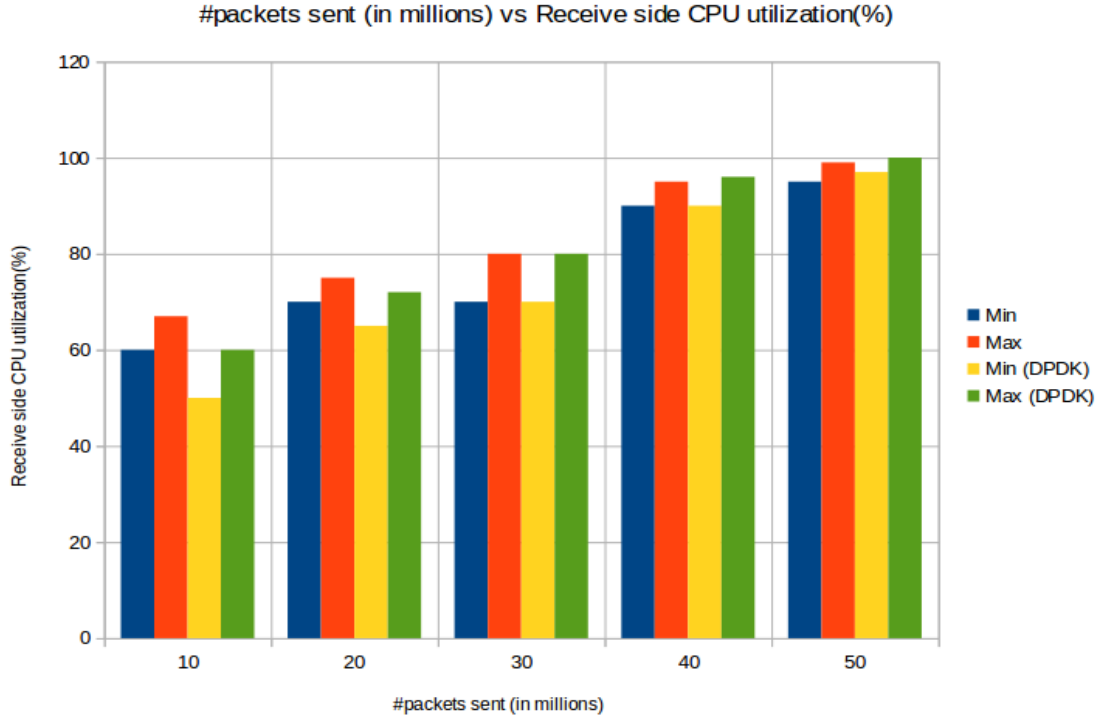


Figure 4.2: No. of Packets sent vs Receive side CPU utilization

Figure 4.2 shows CPU utilization of VM with DPDK is slightly lower than VM without DPDK for initial cases.

Table 4.1 shows the maximum achievable throughput on our system setup.

	Maximum achievable Throughput
VM with DPDK	34 Gbps
VM without DPDK	9 Gbps

Table 4.1: Performance results

Chapter 5

Conclusion

Experiment results shows that using DPDK inside VM with SRIOV enabled host NIC gives more than 50% increase in the performance compare to VM without DPDK.

References

- [1] Dominik Scholz. A look at intel's dataplane development kit http://www.net.in.tum.de/fileadmin/TUM/NET/NET-2014-08-1/NET-2014-08-1_15.pdf, 2014.
- [2] Intel. Dpdk user guide http://dpdk.org/doc/guides/linux_gsg/index.html.
- [3] Intel. Dpdk programmer's guide http://dpdk.org/doc/guides/prog_guide/index.html.
- [4] Intel. Dpdk l2 forwarder http://dpdk.org/doc/guides/sample_app_ug/l2_forward_real_virtual.html.
- [5] Intel. Dpdk pktgen repository <http://www.dpdk.org/browse/apps/pktgen-dpdk/>.
- [6] Udp server example <http://www.tenouk.com/Module40c.html>.